

LANDSLIDE MEDICAL EVACUATION ROUTE OPTIMISER



Dijkstra's Shortest Path Algorithm / Graph Paradigm

Scenario: Post-Landslide Ambulance Routing, Hulu Selangor, Malaysia

KHAIROL RHAZMI BIN FAKHRI (223804)

MUHAMMAD DANIAL BIN WAN NORAHSAM (222448)

DANIEL IQBAL BIN MOHD MARJAN (223749)

SHAH MUHD. ASH-SYAKIR BIN SHAHRIL (SA00339)

CSC4202 Design and Analysis of Algorithms | Semester 2, 2025/2026 | Group: 4

The Disaster



3 Days of Heavy Rain

Northeast monsoon rainfall saturates hillslopes in Hulu Selangor.



Landslide: 14 Nov 2025

200-metre debris flow blocks Jalan Hulu Bernam (the only main highway.)



Medical Emergency: 03:15 AM

Mr. Ahmad bin Ramli (58) suffers a suspected myocardial infarction.



Ambulance Dispatched

Hospital Kuala Kubu Bharu must find an alternate route through logging roads.

THE CHALLENGE

Given the surviving secondary road network,
find the FASTEST route from:



Hospital KKB



Kg. Sungai Lui

to save the patient's life.



Every minute of delay reduces cardiac survival by 7–10%.

Why Is This Hard?

01

Road Network Fragmentation

Only some secondary roads survive. The network is irregular, narrow and partially unknown.

02

Night-Time Conditions

Emergency at 3 AM
-Poor visibility, unlit forest tracks, rain-slicked surfaces.

03

Variable Travel Times

Same distance $\neq$ same time. Logging roads: 15 km/h. Plantation roads: 30 km/h. Weights differ.

04

Multiple Possible Routes

Humans cannot evaluate all route combinations under stress. An algorithm can.

05

Disconnection Risk

If all routes fail, the village is isolated $\rightarrow$ algorithm must detect this and recommend helicopter.

06

Time Constraint

Cardiac reperfusion must occur within 90 minutes of onset. Route must be optimal, not approximate.

Comparing Algorithm Paradigms

Paradigm	Strengths	Weaknesses	Suitable?
Sorting	Simple, fast , running in $O(n \log n)$	Cannot build paths, ignores topology	✗
Divide & Conquer	Breaks problem into subparts	Subproblems share state and not independent	✗
Dynamic Programming	Correct (Bellman-Ford), handles –ve	Belman-Ford runs in $O(V \cdot E)$ For every junction, re-examine every single road and it slow because it repeats work.	yes but slow
Greedy (naïve)	Very fast	Not guarantee globally optimal result	✗
Graph algorithm(Dijkstra's ✓)	Optimal shortest path and run efficiently in $O((V+E)\log V)$ Visit each junction once, check each road once, use a smart queue to stay organised. fast because nothing is repeated.	Requires non-neg weights (✓ satisfied) so the requirement is satisfied.	✓

Dijkstra's Algorithm is the definitive choice -It is purpose-built for non-negative weighted graph shortest paths.

Road Network as a Weighted Graph

NODE LEGEND

- 0 Hospital Kuala Kubu Bharu
- 1 Pekan KKB Town Junction
- 2 Batang Kali Fork
- 3 Sungai Chilling Reserve
- 4 Ladang Ulu Bernam
- 5 Pos Gertak Settlement
- 6 Hulu Bernam Crossroad
- 7 Kampung Sungai Lui

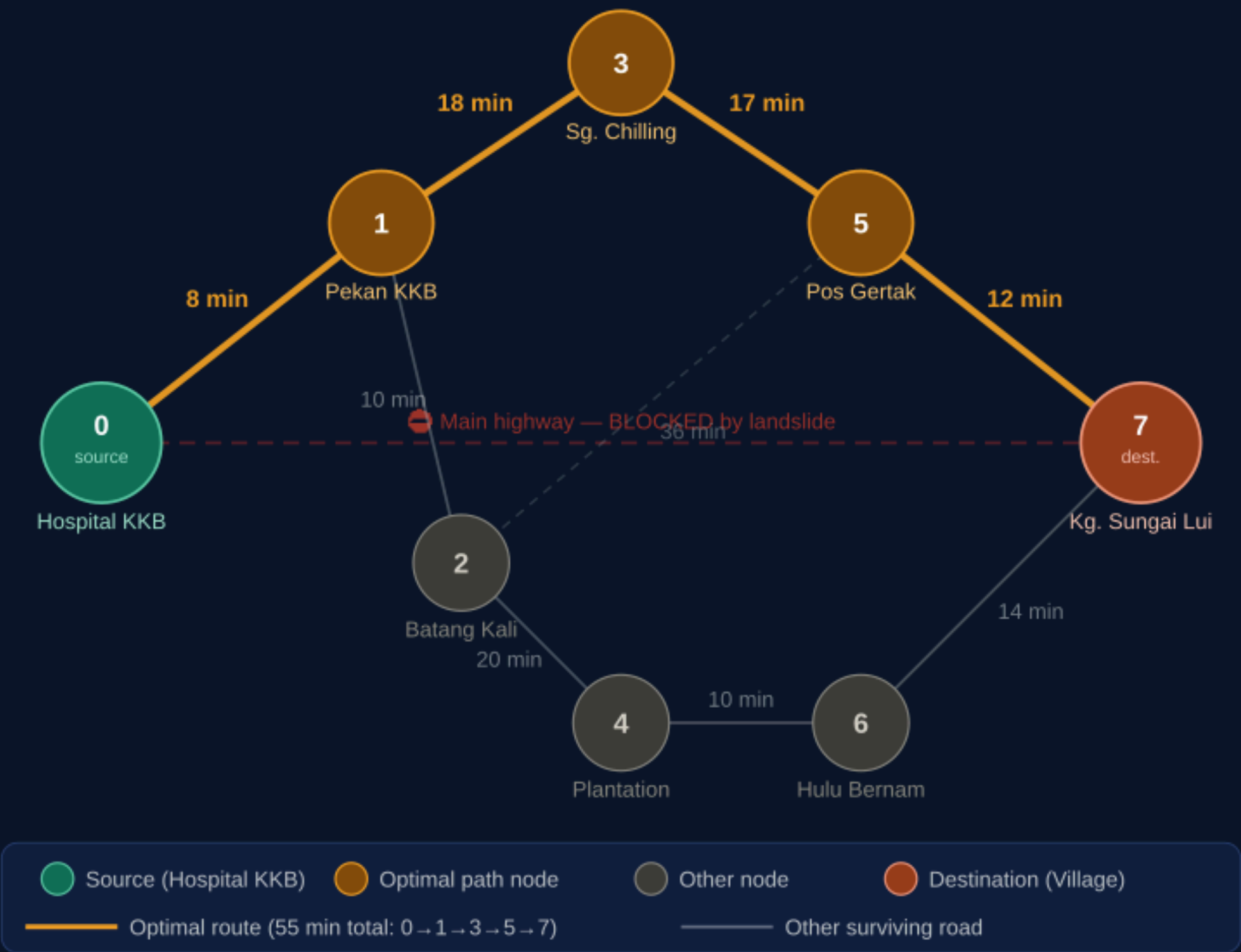
SURVIVING ROAD EDGES (weights = travel time, minutes)

0 – 1	Hospital → Pekan KKB (partial clearance)	8 min
1 – 2	Pekan KKB → Batang Kali Fork	10 min
1 – 3	Pekan KKB → Sg. Chilling Reserve	18 min
2 – 4	Batang Kali → Plantation Estate	20 min
2 – 5	Batang Kali → Pos Gertak (damaged)	36 min
3 – 5	Sg. Chilling → Pos Gertak (forest)	17 min
4 – 6	Plantation → Hulu Bernam Crossroad	10 min
5 – 7	Pos Gertak → Kg. Sungai Lui	12 min
6 – 7	Hulu Bernam → Kg. Sungai Lui	14 min

★ Main Highway (0→7 direct) is BLOCKED by the landslide — omitted from graph.

Post-Landslide Road Network Graph

Post-landslide surviving road network — Hulu Selangor (weights = travel time, minutes)



● Source: Node 0 — Hospital KKB

● Optimal path nodes: 0 → 1 → 3 → 5 → 7 (55 min)

● Destination: Node 7 — Kg. Sungai Lui

Dijkstra's Algorithm — Step-by-Step

PSEUDOCODE

```
dist[s] ← 0;  dist[v] ← ∞  ∀ v ≠ s
PQ.insert(0, source)

while PQ not empty:
  (d, u) ← PQ.extractMin()
  if visited[u]: skip
  visited[u] ← true
  for each (v, w) in adj[u]:
    if dist[u]+w < dist[v]:
      dist[v] ← dist[u] + w
      prev[v] ← u
      PQ.insert(dist[v], v)

path ← reconstruct via prev[]
```

Relaxation: $dist[v] = \min(dist[v], dist[u] + w(u,v))$

EXECUTION TRACE (source = 0)

Step	Extract	Update dist[]
1	Node 0 (0)	dist[1] = 8
2	Node 1 (8)	dist[2]=18, dist[3]=26
3	Node 2 (18)	dist[4]=36, dist[5]=54
4	Node 3 (26)	dist[5]=43 ✓ updated
5	Node 4 (38)	dist[6]=48
6	Node 5 (43)	dist[7]=55
7	Node 6 (48)	dist[7]=62 — no update
8	Node 7 (55)	DESTINATION ✓

Loop Invariant Proof

INVARIANT

At the start of each iteration, for every vertex u in the visited set S : $\text{dist}[u] = \delta(s, u)$ represent the true shortest path distance.

INITIALISATION

$\text{dist}[s] = 0$
mean that at the beginning when source= 0 , mean that the distance to itself is 0, which is obviously to itself. So the algotihm start correctly.

MAINTENANCE

Dijkstra always selects the unvisited node with the smallest distance.
Example:
 $\text{dist}[2] = 18$ min and $\text{dist}[3] = 26$ min
Since Node 2 is closer than Node 3, any path to Node 2 through Node 3 would be longer than 18 minutes. Therefore, Node 2's distance is finalized when it is selected.

TERMINATION

When all nodes have been visited, the algorithm stops.
Since Dijkstra always selects the node with the smallest distance and updates shorter routes whenever they are found, all final distances are correct.

PATH RECONSTRUCTION

Dijkstra first calculates the shortest distance to every node. After reaching the destination, it backtracks through the $\text{prev}[]$ array to reconstruct the route. For this graph, backtracking gives $7 \leftarrow 5 \leftarrow 3 \leftarrow 1 \leftarrow 0$, which becomes $0 \rightarrow 1 \rightarrow 3 \rightarrow 5 \rightarrow 7$.

Asymptotic Analysis

Best Case

$$\Omega((V+E) \log V)$$

Best Case: The algorithm still processes every vertex and edge, so the time complexity remains $O((V + E) \log V)$.

Average Case

$$\Theta((V+E) \log V)$$

Average Case: The algorithm processes all vertices and edges, so the time complexity remains $O((V + E) \log V)$.

Worst Case

$$O((V+E) \log V)$$

In the worst case, Dijkstra has to check every possible connection in the graph. For our graph with only 8 vertices and 9 edges, the total work is very small, so the algorithm runs almost instantly.

Algorithm Output

✓ OPTIMAL ROUTE FOUND — 55 MINUTES



0 → 1	Hospital KKB → Pekan KKB Junction	8 min
1 → 3	Pekan KKB → Sungai Chilling Reserve	18 min
3 → 5	Sungai Chilling → Pos Gertak	17 min
5 → 7	Pos Gertak → Kampung Sungai Lui	12 min
TOTAL	55 minutes — within 90-min cardiac window	✓

Scenario B: If Sg. Chilling road also blocked → Algorithm auto-reroutes via Plantation (62 min) | Scenario C: Village isolated → Alert helicopter

OUTPUT

SCENARIO A: Post-Landslide (Main Highway Blocked)

Source : Node 0 = Hospital Kuala Kubu Bharu ->

Shortest Travel Time : 55 minutes
Optimal Route : 0 -> 1 -> 3 -> 5 -> 7

Segment Breakdown

From -> To	Time (min)	Road Segment
0 -> 1	8	Hospital Kual -> Pekan KKB Town
1 -> 3	18	Pekan KKB Tow -> Sungai Chillin
3 -> 5	17	Sungai Chilli -> Pos Gertak (Or
5 -> 7	12	Pos Gertak (0 -> Kampung Sungai

TOTAL 55 minutes

— All Shortest Distances from Hospital —

NodeID	Time (min)	Location
0	0 min	Hospital Kuala Kubu Bharu
1	8 min	Pekan KKB Town Junction
2	18 min	Simpang Empat Batang Kali Fork
3	26 min	Sungai Chilling Forest Reserve
4	38 min	Ladang Ulu Bernam (Plantation)
5	43 min	Pos Gertak (Orang Asli Settlement)
6	48 min	Hulu Bernam Highlands Crossroad
7	55 min	Kampung Sungai Lui

SCENARIO B: Additional Road Closure (1?3 also blocked)

Source : Node 0 = Hospital Kuala Kubu Bharu ->

Shortest Travel Time : 62 minutes
Optimal Route : 0 -> 1 -> 2 -> 4 -> 6 -> 7

Segment Breakdown

From -> To	Time (min)	Road Segment
0 -> 1	8	Hospital Kual -> Pekan KKB Town
1 -> 2	10	Pekan KKB Tow -> Simpang Empat
2 -> 4	20	Simpang Empat -> Ladang Ulu Ber
4 -> 6	10	Ladang Ulu Be -> Hulu Bernam Hi
6 -> 7	14	Hulu Bernam H -> Kampung Sungai

TOTAL 62 minutes

SCENARIO C: Village Completely Isolated (No Path Exists)

Source : Node 0 = Hospital Kuala Kubu Bharu ->

NO PATH FOUND : village is completely cut off.
Recommend: RMAF/AADK helicopter evacuation.

Conclusion



Scenario

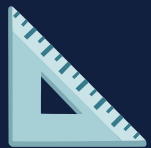
Post-landslide emergency in Hulu Selangor

-A landslide blocked the main highway, forcing the ambulance to navigate through an alternative road network to reach a cardiac patient within a critical time window.



Algorithm Selection

Dijkstra's Algorithm was chosen as the most suitable solution. We turned the map into a network of dots (places) and lines (roads) with travel times on them. Then, we used a smart GPS formula that is guaranteed to find the fastest route.



Correctness & Efficiency

The algorithm is both correct and efficient. We proved this system is 100% accurate and lightning-fast, making it perfect for finding the quickest routes for emergency vehicles in real time.



Complexity

Time Complexity: $O((V + E) \log V)$

The algorithm efficiently processes all vertices and edges using a priority queue, making it suitable for real-time emergency routing applications.



Optimal Result

Shortest Route: $0 \rightarrow 1 \rightarrow 3 \rightarrow 5 \rightarrow 7$
Total Travel Time: 55 minutes



Real-World Impact

Algorithm-guided emergency routing can improve disaster response and save lives.

By automatically identifying the fastest available route after a landslide, Dijkstra's Algorithm enables faster, more reliable, and data-driven emergency decision-making.